EECS/MechE 106A/206A

Robotic Quadruped Obstacle Avoidance in Search and Rescue

Fall 2025

1 Team Information

Name	Background
Denae Cantrell	Denae is a PhD student in the EECS department, whose current project focuses on simulating a frontier-based exploration strategy for a quadruped robot.
Kabir Shah	Kabir is an EECS undergrad student. He has worked on some projects involving autonomous vehicles and distributed ML systems.
Jason Abi Chebli	Jason is a Mechanical Engineering graduate student specializing in Robotics, whose capstone focuses on advanced motion control systems for quadruped robots.
Andrew Tai	Andrew is a Mechanical Engineering graduate student specializing in Robotics with industry software engineering experience.

2 Abstract

Autonomous real-time exploration is essential for operating a robot in an unknown environment, such as search-and-rescue missions. This project develops a real-time frontier-based exploration framework for the Unitree Go2 quadruped. The goal of this project is as follows: travel a given distance in a specified direction without prior knowledge of the environment. First, we integrate core automation systems including SLAM and the Navigation2 stack, which will enable obstacle detection. Then, we design an exploration architecture consisting of three essential ROS2 nodes that: (i) identify spatial boundaries between known and unknown regions in a map (sensing), (ii) computes a utility score that balances effort and goal-directed progress (planning), and (iii) selects frontier goals based on utility scores and interfaces with Nav2 for execution (actuation). The stack is tested on hardware across three different obstacle courses, where metrics are considered as total time and deviation from a provided path.

3 Project Description

Our goal is to develop an autonomous quadruped robot capable of real-time obstacle detection and collision-free navigation for deployment in search-and-rescue environments. The system will enable reliable path planning and maneuvering around debris, uneven surfaces, and unpredictable obstacles.

Each Go2 quadruped robot is equipped with LiDAR, depth cameras, and inertial measurement units (IMUs) for environmental perception. Our system architecture integrates onboard computing, sensor fusion, and motion control within a layered architecture consisting of perception (sensor data processing), planning (path generation and decision-making), and control (motion execution). ROS2 (Robot Operating System) will be used for communication and module integration, with real-time control algorithms implemented in Python.

The perception (sensing) layer continuously gathers spatial data through LiDAR and depth sensors to build a 3D map of the environment. This data is processed using point-cloud algorithms to detect and localize obstacles. The planning layer uses this information to determine the safest and most efficient path around detected obstacles through dynamic trajectory optimization. Finally, the actuation layer converts planned motion commands into joint-level control signals for the quadruped's legs, enabling smooth and stable locomotion over complex terrain.

Testing will be conducted in controlled indoor environments with static and dynamic obstacles. Success will be measured by the ability of the quadruped to detect obstacles autonomously, plan alternative routes, and complete navigation without collisions or instability. Key performance metrics include navigation time, number of obstacle contacts, and avoidance accuracy. Realistic goals include reliable obstacle detection, basic avoidance maneuvers, and successful path completion in static environments. Reach goals include outdoor operation with uneven terrain and debris, and dynamic obstacle avoidance.

4 Tasks

1. **Setup.** We will deploy Go2 with pre-existing Unitree Go2 ROS2 environment.
   (a) **Hardware setup.** Establish reliable ROS2 communication between the workstation and the Unitree Go2 robot. [by 10/30]
   (b) **Software setup.** Launch the provided Unitree ROS2 drivers and robot state publishers (pre-existing). Confirm URDF visualization in RViz, including correct TF frames and joint motion. [by 11/03]
   (c) **Teleoperation Control.** Enable manual driving capability (joystick control), and validate safe locomotion on physical Go2 hardware. Provides a baseline for mobility testing. [by 11/07]
2. **Build autonomy framework.** We will integrate existing autonomy frameworks with Go2, including SLAM
   (a) **SLAM.** Integrate SLAM for online mapping, enabling real-time mapping and localization. This will enable the robot to track its position and orientation within the map. [by 11/10]
   (b) **Navigation2 stack**. Launch Navigation2 (Nav2) stack; utilize it to plan a path from one pose to another. [by 11/14]
   (c) **Incorporating LiDAR.** Nav2 provides methods for 2D costmap generation. Since it allows costmap customization, we will design a costmap that penalizes frontiers next to obstacles. By incorporating LiDAR, a dynamic costmap is generated based on elevation data. [by 11/18]
3. **Develop frontier exploration and ranking modules.** We will design and implement a real-time exploration framework that identifies, evaluates, and select frontier goals.
   (a) **Develop frontier detection node:** The purpose of this node is to provide spatially valid exploration targets. It will detect frontiers, which are boundaries between known and unknown areas in the map, and publish candidate frontier goal locations, which will be visualized as markers in RViz. It is noted that the costmap designed from Nav2 will be utilized as a filter so that only *safe* frontiers are selected as candidates. [by 11/22]
   (b) **Develop frontier ranking node:** We will score frontiers based on distance effort with goal direction bias. This score is obtained from a predefined utility function, which will minimize travel cost and goal bias (frontiers in the direction of the goal are preferred). The node will output scores of the frontier goal candidates. [by 11/26]
   (c) **Develop high-level exploration controller:** This node will take a high-level mission input (distance and direction) and manages real-time exploration progress. It takes the frontier scores from the frontier ranking node, and selects the most beneficial frontier to pursue. The node interfaces with Nav2 to execute each selected frontier goal. [by 11/30]
4. **Testing the final autonomy stack.** We will create multiple obstacles courses to confirm that the robot and ROS2 stack operate as expected.
   (a) **Course construction.** Build 3 test obstacle course using common household items. Mark the floor plans after the course is created for repeatability. [by 12/04]

(b) **Full system testing on course.** Run the final integrated autonomy stack, and collect metrics such as distance progressed in goal direction, successful frontier identification, and time to complete the mission. We will repeat at least 10 trials on a single course for more statistical significance. We will video each trial. [by 12/10]

(c) **Final report.** We will create a final report/website incorporating the final workflow and experimental data. [by 12/19]

5 Bill of Materials

5.1 Use of Lab Resources

No lab resources will be required for this project.

5.2 Other Robotic Platforms

Item	Quantity	Owner/Location
Unitree Go2 Air	1	Prof. Shankar Sastry

5.3 Other Purchases

No other purchases will be required for this project.

6 Other

Note that others in Prof. Sastry's lab have worked with Go2 before, and can act as resources for our group to help setup the robot.

Navigation2 Github: https://github.com/ros-navigation/navigation2

Package	Reason
nav2_bringup	Launch files and examples
nav2_costmap_2d	Local and global 2D costmap generation (we will customize this)
nav2_controller	Local path following and velocity commands
nav2_planner	Global path planning
nav2_msgs	Give action messages (i.e., NavigateToPose)
nav2_bt_navigator	Executes navigation goals
nav2_map_server	Saving and loading the map (saving is important for results)

Table 1: List of pre-existing Navigation2 packages used in this project.